

YOLOv8 TRAINING

Complete Study Guide

Object Detection | Deep Learning | Computer Vision

Sign Language Detection Project — Supplementary Material

Contents

#	Topic	Page
1	What is Object Detection?	2
2	How YOLOv8 Works	3
3	Training Metrics Explained	4
4	Loss Functions Deep Dive	6
5	Transfer Learning	8
6	Overfitting vs Underfitting	9
7	Data Augmentation	10

8	Model Evaluation	11
9	Hyperparameters Guide	13
10	Your Project — Results Analysis	14

1. What is Object Detection?

Object detection is a computer vision task where a model must answer two questions simultaneously for every object in an image: **WHERE** is it (bounding box) and **WHAT** is it (class label). This is harder than image classification, which only answers "what is the dominant object in this image?" with no location information.

Three levels of visual recognition:

Task	Input	Output	Example
Image Classification	Full image	One label	"This is a cat"
Object Detection	Full image	Boxes + labels	"Cat at (x,y,w,h)"
Segmentation	Full image	Pixel masks	"These pixels = cat"

Bounding Boxes

A bounding box is a rectangle described by four numbers. YOLOv8 uses the normalized centre-point format internally:

```
x_center, y_center, width, height (all values between 0 and 1)
```

Example: a hand occupying the middle 40% of a 640x640 image would be stored as **0.5, 0.5, 0.4, 0.4** — centre at (50%, 50%), spanning 40% width and 40% height.

Why normalize? Storing values as fractions of image dimensions means the same label works at any resolution — 640x640 or 1280x720. You don't need separate label files for different image sizes.

How a Prediction Becomes a Detection

YOLOv8 outputs thousands of candidate boxes simultaneously. Most are empty or wrong. Two filters clean them up:

- 1. Confidence threshold** — discard boxes with confidence below a cutoff (you use 0.5 = 50%). Boxes below this are treated as "no detection."
- 2. Non-Maximum Suppression (NMS)** — when multiple boxes overlap for the same object, keep only the highest-confidence one. Removes duplicate detections.

2. How YOLOv8 Works

YOLO stands for "You Only Look Once" — named because it processes the entire image in a single forward pass, unlike older two-stage detectors (like the TFOD API you used previously) that first propose regions then classify them.

Three-Stage Architecture

Stage	Component	What it does
1 — Backbone	CSP-Darknet + C2f blocks	Extracts features from the image — edges, textures, shapes, patterns
2 — Neck	FPN + PAN (feature pyramid)	Combines features from different scales — helps detect both small and large hands
3 — Head	Decoupled detection head	Predicts box coordinates and class probabilities simultaneously

Your model had **170 layers** and **25.8 million parameters** — each parameter is a floating-point number that gets adjusted during training. The goal of training is to find the best values for all 25.8 million numbers so the model detects your 24 hand signs correctly.

YOLOv8 Model Sizes

Model	Parameters	Speed (T4)	mAP COCO	Use case
yolov8n (nano)	3.2M	Fastest	37.3	Edge devices, real-time
yolov8s (small)	11.2M	Fast	44.9	Good balance — you used this
yolov8m (medium)	25.9M	Medium	50.2	Higher accuracy
yolov8l (large)	43.7M	Slow	52.9	Research/high-accuracy
yolov8x (xlarge)	68.2M	Slowest	53.9	Maximum accuracy

Your choice: yolov8s (sign_lang_best.pt) is the right size for your project. Fast enough for real-time webcam inference on a GTX 1650, accurate enough for 24-class hand sign detection. Using yolov8m would give ~1% more accuracy but 2x slower inference.

3. Training Metrics Explained

Every epoch prints a row of numbers. Here is exactly what each column means, what a good value looks like, and what to do if it goes wrong.

Epoch

One complete pass through your entire training dataset. If you have 1024 training images and `batch_size=16`, one epoch = 64 batches ($1024 / 16 = 64$ steps).

Each epoch the training data is reshuffled randomly, so the model sees images in a different order — this prevents it from memorising the sequence rather than the content.

Your result: 21 epochs completed. Best performance was at epoch 1 because you fine-tuned from an already well-trained model. Early stopping triggered correctly at epoch 21.

GPU_mem

The amount of GPU VRAM (video RAM) in use during training. Measured in gigabytes.

If this approaches your GPU's limit, training will crash with an Out Of Memory (OOM) error. The fix is always to reduce `batch_size`.

GPU	VRAM	Safe batch_size (640px)
GTX 1650 (yours, local)	4 GB	batch=4
T4 (Colab free)	14.9 GB	batch=16
A100 (Colab Pro)	40 GB	batch=64

Instances

The number of labeled bounding boxes across all images in the current batch. This varies because different images contain different numbers of visible hands (some may have partial hands, some very clear single hands).

Your range was 24-40 per batch — this is normal variation and does not affect results.

Size

The input image resolution fed to the model. YOLOv8 resizes all images to this square size before processing. Standard values: 320, 416, 512, 640, 1280.

imgsz	Speed	Accuracy	Use when
320	Fastest	Lower	Very fast inference needed, small objects not critical
640	Balanced	Good	Default — almost always the right choice
1280	Slow	Best	Small/detailed objects, research use

4. Loss Functions Deep Dive

Loss is a number that measures how wrong the model is. The training loop tries to minimise total loss by adjusting the model's 25.8 million parameters. Lower loss = better predictions.

Key concept: Loss is computed on training data. Validation loss is computed on held-out data the model never trained on. If training loss drops but validation loss rises, the model is overfitting — memorising training images instead of learning general patterns.

box_loss — Location Error

Measures how inaccurate the predicted bounding box position is. YOLOv8 uses CloU (Complete Intersection over Union) loss for this.

CloU considers three things simultaneously:

- Overlap area between predicted and actual box

- Distance between box centres

- Aspect ratio difference (wide vs tall boxes)

Your values: epoch 1 = 0.354, epoch 21 = 0.381. The slight increase during fine-tuning is normal — the model is adjusting to your specific hand images. Values below 0.5 are excellent.

cls_loss — Classification Error

Measures how often the model predicts the wrong class. Uses Binary Cross-Entropy loss applied independently to each class.

Binary Cross-Entropy asks: for each class (A, B, C... Y), what is the probability the model assigns to the correct class? If it assigns 0.95 probability to the correct class, cls_loss is very low. If it assigns 0.3, cls_loss is high.

Your values: started at 0.342, general downward trend. Final overall accuracy 99.4% confirms cls_loss is very well controlled.

dfl_loss — Precision Refinement

DFL stands for Distribution Focal Loss. It is a secondary refinement on top of box_loss that asks: are the four box edges (left, top, right, bottom) placed precisely on the exact boundary of the hand, or slightly off?

DFL models each edge position as a probability distribution across nearby pixels, rather than a single prediction. This produces more precise box boundaries.

Your values: 0.91-0.96 throughout. These are consistently excellent — your hand images have clear, well-defined edges that the model finds easily.

Total Loss Formula

```
total_loss = (box_weight x box_loss) + (cls_weight x cls_loss) + (dfl_weight x dfl_loss)
```

Default YOLOv8 weights: box=7.5, cls=0.5, dfl=1.5. Box loss gets the most weight because finding the object matters more than perfectly classifying it in early training.

Loss component	Default weight	Your epoch 1 value	Contribution
box_loss	7.5x	0.354	2.655 (largest)
cls_loss	0.5x	0.342	0.171 (smallest)
dfl_loss	1.5x	0.910	1.365 (medium)
Total loss	—	—	~4.19 weighted

5. Transfer Learning

Transfer learning is the technique of starting training from weights that were previously trained on a large dataset, rather than from random numbers. This is why your model achieved 99.4% accuracy after just 21 epochs.

Why it works

Neural network layers learn to detect progressively more complex features:

Early layers: edges, colour gradients, simple textures — universal across all images

Middle layers: shapes, curves, object parts — partially universal

Late layers: specific objects — changes for each new task

When you fine-tune from `sign_lang_best.pt`, the early layers already know how to detect edges and hand shapes. You only need to teach the late layers the difference between the new M and N hand positions. This requires far less data and training time.

Your Transfer Learning Chain

Stage	Starting weights	Trained on	Result
1 — COCO pretraining	Random (Xavier init)	80 classes, 118k images	yolov8s.pt
2 — Your first training	yolov8s.pt	A-Y, ~50 images/class	sign_lang_best.pt
3 — Fine-tuning (latest)	sign_lang_best.pt	A-Y + new M/N images	AY_v2/best.pt

COCO (Common Objects in COntext) is a benchmark dataset with 118,000 images and 80 object categories. The `yolov8s.pt` you downloaded already "knows" how to detect people, hands, and common objects from this training. Your fine-tuning specialised this general knowledge for ASL fingerspelling specifically.

6. Overfitting vs Underfitting

	Underfitting	Good fit	Overfitting
Training loss	High	Low	Very low
Validation loss	High	Low (similar to train)	High (diverges from train)
Real-world accuracy	Poor	Good	Poor on new images
Cause	Too few epochs / model too small	Correct training	Too many epochs / too little data
Fix	Train longer / larger model	—	Early stopping / more data / dropout

Early Stopping

Early stopping monitors validation accuracy. If it does not improve for **patience** consecutive epochs, training stops and restores the best weights.

patience=20 in your training, metric=mAP50. Best mAP50 was at epoch 1. Stopped at epoch 21 (1 + 20 = 21). Best weights automatically saved as best.pt.

Dropout

Dropout is a regularisation technique that randomly "turns off" a fraction of neurons during each training step. This forces the model to learn redundant representations — it cannot rely on any single neuron always being available.

Your LSTM model uses Dropout(0.2) and Dropout(0.3). YOLOv8 uses a different technique called DropBlock which works similarly but operates on spatial blocks of features.

7. Data Augmentation

Data augmentation creates modified versions of training images on-the-fly. The model sees a slightly different version of each image every epoch, making it more robust to real-world variations in lighting, angle, and position.

YOLOv8 applies augmentation automatically during training. Here are the key ones:

Augmentation	What it does	Helps with
Horizontal flip (p=0.5)	Mirrors image left-right	Left vs right hand variations
HSV hue jitter (0.015)	Slight colour shift	Different skin tones, lighting
HSV saturation (0.7)	Changes colour intensity	Washed-out or vivid lighting
HSV value (0.4)	Brightness adjustment	Dark rooms, bright windows
Scale (0.5)	Zooms in/out randomly	Different hand distances from camera
Translate (0.1)	Shifts image position	Hand near edge of frame
Mosaic (1.0)	Combines 4 images in grid	Background variety, multi-scale
Erasing (0.4)	Masks random patches	Partial occlusion (sleeve, other hand)
Blur (p=0.01)	Slight Gaussian blur	Motion blur during fast signing
CLAHE (p=0.01)	Contrast enhancement	Low-contrast images

Mosaic augmentation is one of YOLO's most important innovations. It combines four training images into a 2x2 grid, forcing the model to detect objects in unusual contexts and at multiple scales simultaneously. This dramatically improves performance with small datasets like yours.

8. Model Evaluation

Precision and Recall

These two metrics come from the confusion matrix and measure different types of errors:

Metric	Formula	Meaning	Bad value means...
Precision	$TP / (TP + FP)$	Of all detected M's, how many were actually M?	Too many false alarms
Recall	$TP / (TP + FN)$	Of all actual M's in images, how many were found?	Missing real detections
F1 Score	$2 \times P \times R / (P+R)$	Harmonic mean of precision and recall	Imbalanced P vs R

TP = True Positive (correctly detected), **FP** = False Positive (detected something that wasn't there or wrong class), **FN** = False Negative (missed a real hand sign).

IoU — Intersection over Union

IoU measures how much your predicted bounding box overlaps with the ground-truth box. It is the primary criterion for deciding if a detection counts as correct.

$$\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$$

IoU value	Interpretation
1.0	Perfect — boxes match exactly
0.7+	Very good — boxes closely aligned
0.5	Threshold used in mAP50 — boxes overlap by at least half
0.3	Poor — boxes loosely aligned
0.0	No overlap — completely wrong location

mAP50 and mAP50-95

mAP (mean Average Precision) is the standard object detection metric. It summarises the precision-recall trade-off across all confidence thresholds and all classes.

mAP50: Counts a detection as correct if IoU > 0.5. Lenient threshold.

mAP50-95: Averages mAP across IoU thresholds from 0.5 to 0.95 in steps of 0.05. Much stricter — requires boxes to be nearly perfectly placed.

Metric	Your value	Industry benchmark	Interpretation
--------	------------	--------------------	----------------

mAP50	99.4%	50%+ acceptable, 90%+ excellent	Outstanding
mAP50-95	96.8%	40%+ acceptable, 70%+ excellent	Outstanding
Precision	98.9%	80%+ good	Very high
Recall	98.3%	80%+ good	Very high

9. Hyperparameters Guide

Hyperparameters are settings you choose before training starts. Unlike model parameters (weights), hyperparameters are not learned — you set them manually. Choosing the right values significantly affects training speed and final accuracy.

Hyperparameter	Your value	Effect of increasing	Effect of decreasing
epochs	100	More training passes, risk of overfitting	Less trained, may underfit
batch	16 (Colab)	Smoother gradients, needs more VRAM	Noisier updates, less VRAM
imgsz	640	Higher accuracy, much slower	Faster, lower accuracy
lr0 (initial LR)	0.01 (auto)	Faster but unstable convergence	Slower convergence
lrf (final LR)	0.01	Ends training at higher LR	More fine-tuning at end
patience	20 (yours)	Trains longer before stopping	Stops too early
conf (inference)	0.5 (yours)	Fewer but more certain detections	More detections, more false alarms
iou (NMS)	0.7	Allows more overlapping boxes	Aggressively removes duplicates

Learning rate (lr0) is the most important hyperparameter. Too high: model overshoots the optimal weights and loss oscillates. Too low: training is very slow. YOLOv8 uses "optimizer=auto" which selects AdamW with an automatically computed LR based on batch size — this is why you see lr=0.000357 in your training log.

10. Your Project — Results Analysis

Model Performance Summary

After fine-tuning from `sign_lang_best.pt` with additional M and N class images, all 24 classes achieved 99%+ mAP50. Here are the final per-class results:

Class	mAP50	Precision	Recall
A	97.9%	100%	64.5%
B	99.1%	93.4%	94.0%
C	99.5%	98.9%	100%
D	99.5%	98.8%	100%
E	99.5%	99.3%	100%
F	99.5%	98.7%	100%
G	99.5%	99.0%	100%
H	99.5%	99.1%	100%
I	99.5%	98.8%	100%
K	99.5%	99.0%	100%
L	99.5%	99.2%	100%
M	99.5%	99.2%	100%
N	99.5%	99.4%	100%
O	99.5%	99.2%	100%
P	99.5%	99.1%	100%
Q	99.5%	99.5%	100%
R	99.5%	98.8%	100%
S	99.5%	98.9%	100%
T	99.5%	98.8%	100%
U	99.5%	98.9%	100%
V	99.5%	98.8%	100%
W	99.5%	98.8%	100%
X	99.5%	100%	100%
Y	99.5%	99.1%	100%
OVERALL	99.4%	98.9%	98.3%

Class A — Low Recall Explained

Class A shows 100% precision but only 64.5% recall. This means: when the model says "this is A", it is always right (precision=100%). But it misses some actual A gestures (recall=64.5%). This is likely because A and S look very similar — closed fist with slight thumb differences. Collecting more varied A images would fix this.

Precision vs Recall trade-off: Increasing the confidence threshold raises precision (fewer false positives) but lowers recall (more misses). Decreasing it does the opposite. Your conf=0.5 setting is a balanced default. For a communication aid, higher recall (missing fewer letters) is usually more important than precision.

Next Steps for Improvement

Action	Expected improvement	Priority
Collect more varied A images (different lighting, angles)	A recall 64% -> 90%+	High
Upgrade to YOLOv11 (same API, improved backbone)	+1-2% overall mAP	Medium
Add common phrase detection (Hello, Thank You)	New feature	Medium
Text-to-speech integration	New feature	Medium
TFLite export for mobile deployment	New platform	Low